

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 05 -

MONITORAMENTO E MANUTENÇÃO

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	25
O QUE VOCÊ VIU NESTA AULA?	29
REFERÊNCIAS.....	30

EMSE

O QUE VEM POR AÍ?

Após colocar um modelo de Machine Learning em produção, o trabalho está longe de estar terminado. Modelos de ML não são sistemas estáticos – seu desempenho inevitavelmente tende a se degradar ao longo do tempo devido a mudanças sutis (ou nem tão sutis) no ambiente de produção. Nesta aula, você aprenderá como monitorar continuamente modelos implantados e realizar a manutenção proativa para garantir que eles continuem entregando valor com confiabilidade. Discutiremos como detectar sinais de degradação de performance, como definir métricas de acompanhamento (desde acurácia preditiva até latência de resposta e uso do modelo) e quais ferramentas e práticas utilizamos para observar, diagnosticar e adaptar modelos em produção.

Para “quebrar o gelo”, imagine um cenário: você treinou um modelo de detecção de doença cardíaca com alta acurácia em laboratório. No entanto, após alguns meses em produção, ele começa a errar diagnósticos com mais frequência. O que estaria acontecendo? Poderia ser que os padrões nos dados mudaram, ou seja, seu modelo pode estar enfrentando novas situações e padrões nos dados sem que ninguém tenha percebido ainda. Nesta aula, veremos como identificar rapidamente estes e outros problemas – antes que causem impacto negativo – por meio de monitoramento de métricas, além de estratégias de re-treinamento e versionamento do modelo para mantê-lo sempre atualizado e confiável. Prepare-se para entrar no mundo do MLOps pós-implantação, onde manter modelos funcionando bem é tão desafiador quanto desenvolvê-los!

HANDS ON

Neste hands on, daremos sequência prática ao caso da última aula, em que implantamos um modelo de detecção de doenças cardíacas usando um endpoint via REST API. Agora, nosso foco será implementar um processo de monitoramento de métricas desse modelo em produção e definir critérios para retreino quando necessário. Em outras palavras, vamos instrumentar o modelo implantado para coletar indicadores de performance ao longo do tempo e reagir quando esses indicadores apontarem degradação.

Veremos na Seção Saiba Mais, como podemos adaptar a aplicação para registrar métricas em tempo real durante as inferências do modelo. Em uma aplicação de modelo de predição, como o de doença cardíaca, o monitoramento envolve o registro de dados a cada requisição de inferência. Isso inclui o payload (dados de entrada usados na chamada), a latência (tempo de resposta da inferência) e métricas personalizadas sobre a distribuição dos valores de entrada, permitindo identificar proativamente quaisquer alterações nos padrões dos dados que chegam ao modelo.

SAIBA MAIS

Vamos iniciar pelo monitoramento das requisições. Para isso, utilizaremos logs estruturados que serão coletados diretamente da aplicação Flask API que criamos anteriormente. Esses logs capturam dados essenciais sobre as interações com o modelo, oferecendo uma solução robusta para o acompanhamento e a análise do tráfego de inferência. A coleta desses logs é crucial para:

- Monitoramento de Desempenho e Qualidade
 - Registra a carga (payload) da solicitação, a resposta (incluindo o resultado da inferência) e metadados importantes, como latência e status HTTP da solicitação.
 - Permite a análise posterior de desvios (data drift) nos dados de entrada e saída, identificando quando o modelo pode estar performando mal em produção devido a mudanças no ambiente ou nos dados.
- Debugging e Auditoria
 - Funciona como um log histórico e estruturado de todas as interações com o modelo.
 - Facilita a depuração de erros, permitindo rastrear exatamente o que foi enviado e recebido em uma requisição com falha.
 - Atende a requisitos de auditoria e conformidade, fornecendo um registro imutável de todas as previsões.
- Refinamento e Retreinamento de Modelos
 - Os dados capturados nos logs podem ser usados para identificar exemplos onde o modelo está incerto ou cometendo erros.

Estes insights alimentam ciclos de treinamento, permitindo que as versões futuras do modelo sejam melhoradas com dados de produção relevantes.

Ao configurar a aplicação para registrarmos as requisições de inferência, iremos estruturar um padrão de coleta de dados que permite posteriormente utilizar os dados para análises da qualidade das respostas do modelo ao longo do tempo.

Cada requisição ao endpoint irá registrar as informações necessárias, conforme a tabela 1.

CAMPO	DESCRIÇÃO
timestamp	O momento em que a requisição foi processada.
request_id	Um ID único para a requisição.
request_payload	O conteúdo exato enviado na requisição (JSON, texto, etc.).
response_payload	O conteúdo da resposta do modelo.
status_code	O código HTTP da resposta.
latency_ms	A latência total da requisição em milissegundos.

Tabela 1 - Estrutura Típica dos Dados Capturados
Fonte: Elaborado pelo autor (2026)

Adicionaremos na nossa aplicação Flask um método para coleta dos logs de requisição utilizando a biblioteca Python logging. A biblioteca logging é um módulo padrão do Python que oferece um sistema flexível para o registro de eventos de programas. É uma ferramenta essencial para debugging, auditoria e monitoramento de aplicações em produção, especialmente em cenários de Machine Learning Operations (MLOps) como o monitoramento de requisições de inferência.

O sistema de logging permite que você categorize as mensagens com base na sua importância. Para o monitoramento de requisições em uma API Flask, a biblioteca

é configurada para capturar `request_payload`, `response_payload`, `latency_ms` e outros dados cruciais. Ao usar um `Formatter` ou um `Handler` customizado que serializa esses dados em JSON, garantimos que os logs sejam facilmente legíveis e que possam ser consultados por sistemas de monitoramento (como ELK Stack, Splunk ou Prometheus/Grafana).

```
# -----  
# Configuração de logging estruturado  
# -----  
logger = logging.getLogger("request_logger")  
logger.setLevel(logging.INFO)  
_log_path = os.getenv(LOG_ENV_VAR, DEFAULT_LOG_PATH)  
if not logger.handlers:  
    fh = logging.FileHandler(_log_path)  
    fh.setLevel(logging.INFO)  
    formatter = logging.Formatter('%(message)s')  
    fh.setFormatter(formatter)  
    logger.addHandler(fh)
```

Código-fonte 1 - Configuração de logging na aplicação (Python)
Fonte: Elaborado pelo autor (2026)

Apesar de estarmos implementando o monitoramento via logging estruturado na nossa API Flask, é importante saber que outras plataformas MLOps corporativas oferecem soluções que simplificam drasticamente esse processo. Por exemplo, se estivéssemos utilizando a Databricks (como fizemos em exemplos anteriores para desenvolvimento e tracking de modelos), o recurso de Inference Tables poderia automatizar grande parte da coleta de dados. As Inference Tables são uma ferramenta poderosa que podem ser configuradas nos endpoints para capturar automaticamente os dados de entrada (`request payload`), as previsões de saída (`response payload`) e metadados de requisição (como `timestamp` e `latência`) em modelos servidos pela plataforma (Model Serving).

- **Coleta Automática:** Capturam payloads de entrada e saída e metadados de latência em tempo real, eliminando a necessidade de instrumentar manualmente o código da aplicação com logging detalhado.

- **Armazenamento Estruturado:** Os dados são persistidos diretamente no Delta Lake em formato de tabela, de forma otimizada, o que facilita a consulta, análise e o uso imediato para data drift detection e model retraining.
- **Esquema Padrão:** A tabela segue um esquema pré-definido, incluindo campos como timestamp, request_payload, response_payload, status_code, entre outros, garantindo consistência e padronização dos dados de inferência, prontos para uso em dashboards de monitoramento.

O uso de Inference Tables acelera significativamente o tempo de implementação do monitoramento de produção, permitindo que as equipes se concentrem mais na análise dos insights gerados do que na infraestrutura de coleta de dados.

Voltando ao contexto da adaptação que fizemos na nossa aplicação Flask API, agora estamos coletando os dados de requisições realizadas em um arquivo de log que armazena os dados em formato .json:

```
{
  "timestamp": "2025-11-20T20:04:43.027997Z",
  "request_id": "9773890b-082b-43cd-bfce-e852f6426e6e",
  "request_payload": {
    "age": 63,
    "sex": 1,
    "cp": 3,
    "trestbps": 145,
    "chol": 233,
    "fbs": 1,
    "restecg": 0,
    "thalch": 150,
    "exang": 0,
    "oldpeak": 2.3,
    "slope": 0,
    "ca": 0,
    "thal": 1
  },
  "response_payload": {
    "predictions": [1],
    "probabilities": [0.5373601118205007]
  },
  "status_code": 200,
  "latency_ms": 14.363
}
```

Figura 1 - Exemplo de registro de uma requisição coletada no arquivo de log
Fonte: Elaborado pelo autor (2026)

Precisamos agora analisar dados que serão coletados para identificarmos se os dados de requisições seguem o mesmo padrão de distribuição de dados que tínhamos no treinamento do modelo ou se houve o efeito de Data Drift.

O Data Drift é um termo fundamental no Machine Learning (ML) que se refere à mudança nas propriedades estatísticas dos dados de entrada (variáveis preditoras) ou na relação entre os dados de entrada e a variável alvo (variável que o modelo tenta prever) ao longo do tempo, após o modelo ter sido treinado e implantado em produção. Quando ocorre Data Drift, o ambiente de produção onde o modelo está sendo aplicado não se assemelha mais ao ambiente em que ele foi treinado. Isso significa que as premissas nas quais o modelo se baseia para fazer previsões precisas não são mais válidas, levando inevitavelmente à degradação do desempenho do modelo em produção.

Como exemplo, imagine que nossa API em produção recebeu centenas de requisições ao longo do tempo, e dada a degradação de qualidade das previsões, apontada por exemplo pelas áreas de negócio, precisamos analisar se existe o efeito de Data Drift nas features ao longo do tempo. Para isso, podemos analisar se a distribuição das variáveis mudou ao longo do tempo, comparando um conjunto de dados de produção recente com um conjunto de dados de referência (dados de treino).

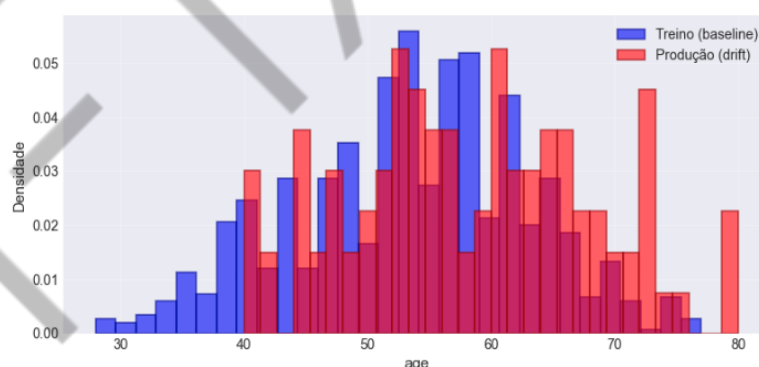


Figura 2 - Análise de Data Drift da variável - age (idade)

Fonte: Elaborado pelo autor (2026)

Podemos perceber pelo gráfico que a distribuição das barras em Azul (dados de treino) tem um comportamento bastante diferente se comparada à distribuição das barras em Vermelho (dados de Produção). Estas grandes diferenças na distribuição

indicam um efeito significativo de Data Drift e podem estar causando impactos negativos nas previsões do modelo.

A Importância de Monitorar o Data Drift

O monitoramento contínuo do Data Drift é uma prática essencial no MLOps. A detecção precoce do desvio de dados permite que a equipe de ML adote medidas corretivas, como:

- **Retreinamento:** retreinar o modelo com os novos dados de produção que refletem a realidade atual.
- **Ajuste de Recursos:** investigar e, se necessário, re-engenhar as características (features) que estão sofrendo desvio.
- **Alertas:** acionar alertas automáticos para notificar os responsáveis quando um limite de desvio é ultrapassado, antes que a degradação da performance cause impactos negativos no negócio.

Monitoramento da Latência do Modelo

Além do desempenho preditivo do modelo (como acurácia, F1-Score etc.) e do desvio (Model e Data Drift), o monitoramento da latência é um componente crítico da manutenção de modelos de Machine Learning em produção, especialmente para aplicações em tempo real. A latência refere-se ao tempo total que leva desde o momento em que uma requisição de inferência é enviada ao modelo até o momento em que a resposta (predição) é retornada. A alta latência, ou o aumento inesperado da latência ao longo do tempo, pode ter um impacto direto e negativo na experiência do usuário e nos resultados de negócio.

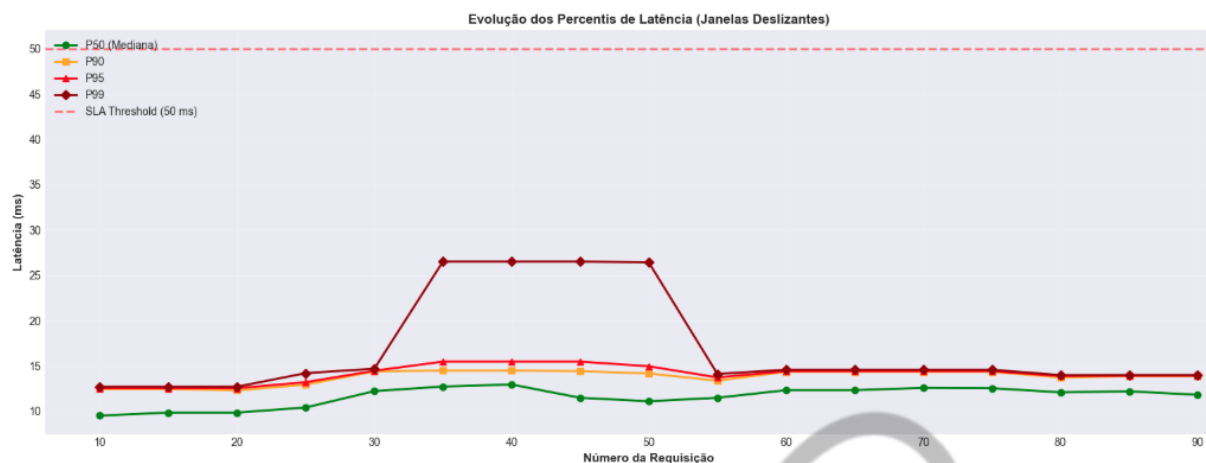


Figura 3 - Evolução dos percentis de Latência para diferentes Números de Requisições
Fonte: Elaborado pelo autor (2026)

A prática de monitorar diferentes percentis na análise de latência é importante para obter uma visão abrangente e realista do desempenho de um modelo em produção, indo muito além da simples média. O P50 (percentil 50), que é sinônimo da mediana, oferece o tempo de resposta típico. Por ser menos sensível a outliers do que a média, o P50 é um ponto de partida para entender o desempenho central do sistema. Contudo, para garantir a experiência dos usuários que enfrentam as piores condições, as métricas de cauda, como o P95 (percentil 95) e o P99 (percentil 99), são indispensáveis no monitoramento. O P95 indica o tempo de resposta máximo que 95% das requisições conseguem atingir, enquanto o P99 revela o tempo máximo de resposta para 99% das requisições.

Monitorar o P95 e, principalmente, o P99 (o "pior caso" de latência para a maioria dos clientes) permite que as equipes identifiquem e corrijam gargalos que só se manifestam sob estresse ou em condições específicas de infraestrutura. Um aumento repentino no P99 pode sinalizar um problema de alocação de recursos ou a necessidade de otimizar a lógica de serving. Ao estabelecer limites de alerta rigorosos para esses percentis de latência mais altos, as equipes garantem a satisfação do cliente e o cumprimento de Acordos de Nível de Serviço (SLAs), assegurando a confiabilidade e a rapidez do sistema para virtualmente todos os usuários.

Outros Itens Importantes no Monitoramento de Modelos

Além de Data Drift e latência, um monitoramento completo de modelos em produção deve abarcar uma série de outros indicadores e práticas essenciais para garantir a robustez, a equidade e o alinhamento com os objetivos de negócio.

Monitoramento da Qualidade Preditiva (Model Drift): embora o Data Drift aponte mudanças nos dados de entrada, o Model Drift (ou Performance Drift) monitora diretamente a degradação do desempenho preditivo do modelo ao longo do tempo. A chave para o monitoramento da qualidade preditiva é a coleta de ground truth (os dados reais que o modelo tentou prever) após a inferência. Em muitos cenários (como previsão de churn ou detecção de fraude), o resultado real só está disponível horas, dias ou semanas após a predição. É vital ter um mecanismo para reconciliar as predições feitas com seus resultados reais correspondentes para calcular as métricas de qualidade.

Modelos de ML não existem isoladamente; eles impactam métricas de negócio. O monitoramento deve incluir:

- **Impacto no Negócio:** acompanhamento de KPIs (Key Performance Indicators) diretamente influenciados pelas predições do modelo (ex.: aumento de vendas após uma recomendação, redução de fraudes detectadas).
- **Taxa de Uso do Modelo:** frequência e volume de requisições, ajudando a entender a adoção e a necessidade de escalabilidade.
- **Monitoramento de Viés e Equidade (Fairness):** em um contexto crescente de regulamentação e responsabilidade ética, é obrigatório monitorar se o modelo está tomando decisões justas e imparciais.
- **Métricas de Equidade:** análise da acurácia, F1-Score ou outras métricas de desempenho desagregadas por grupos sensíveis (gênero, etnia, localização).
- **Desigualdade nas Predições:** detecção de padrões onde o modelo consistentemente erra ou penaliza desproporcionalmente um subgrupo.

A complexidade do monitoramento de modelos em produção leva muitas empresas a utilizarem plataformas especializadas em monitoramento, como

Prometheus, Grafana, DataDog e soluções de provedores de nuvem (AWS SageMaker, Azure ML, Google AI Platform, Databricks etc.), que automatizam grande parte dos processos de coleta, análise e alerta.

A escolha da ferramenta ideal depende da maturidade da equipe, da infraestrutura de serving (nuvem ou on-premise) e da necessidade de monitoramento especializado de ML. No entanto, a base de qualquer sistema robusto permanece a mesma: a coleta de logs estruturados e a definição clara de métricas de negócio, desempenho e equidade.

Vamos nos aprofundar agora nos conceitos e boas práticas de monitoramento e manutenção de modelos de ML após o deployment. Você verá que garantir a saúde do modelo em produção exige tanto acompanhamento técnico (métricas, logs, alertas) quanto processos de negócio (rotinas de atualização, governança de versões). A seguir, organizamos o conteúdo em subtemas para facilitar.

Por que monitorar modelos em produção?

Assim que um modelo de ML entra em produção, ele fica sujeito a mudanças no mundo real que podem reduzir sua eficácia. Esse fenômeno é frequentemente descrito como drift (“model ou data drift”) e ocorre porque a relação que o modelo aprendeu pode deixar de valer se o contexto de dados mudar. Em outras palavras, modelos “envelhecem”: padrões que antes eram verdadeiros podem mudar e sem intervenção o modelo continuará fazendo previsões com base em padrões antigos.

Estudos da indústria mostram que mesmo tendências sutis nos dados de produção podem alterar radicalmente o comportamento do modelo. Por isso, não basta acompanhar apenas métricas básicas de sistema (como uptime do serviço); é crucial acompanhar continuamente a qualidade preditiva e o contexto dos dados para detectar regressões de desempenho e agir antes que prejudiquem os objetivos de negócio.

Dois problemas correlacionados ilustram bem a necessidade de monitoramento: o skew de treinamento vs. produção e o drift de modelo ou de dados. O training-serving skew ocorre quando há diferenças significativas entre as condições dos dados de treinamento e os dados reais que o modelo vê em produção, levando a resultados inesperados. Já a model drift diz respeito a alterações ao longo do tempo

nos dados ou na relação entre dados e rótulos, causando queda de acurácia e confiabilidade. O drift pode ser dividido em duas principais categorias:

- **Data Drift:** ocorre quando a distribuição dos dados de entrada muda ao longo do tempo, sem alteração na definição do problema em si. Por exemplo, se um modelo foi treinado com exames médicos de um equipamento e em produção os exames vêm de um novo equipamento com características diferentes, os inputs mudaram de perfil. A relação entre entrada e saída permanece a mesma, mas as características dos dados de entrada se deslocaram. Esse tipo de mudança é chamado de drift de atributos/features e pode levar a degradação silenciosa do desempenho preditivo. Em outras palavras, o modelo passa a receber dados “diferentes” do que espera, perdendo acurácia gradualmente.
- **Model Drift:** ocorre quando muda a relação fundamental entre as entradas e o alvo predito. Aqui, a própria definição do que estamos tentando prever pode evoluir. Por exemplo, um modelo de detecção de fraude financeira pode começar a falhar se os fraudadores adotarem novas táticas; aquilo que caracteriza uma transação fraudulenta mudou ao longo do tempo. Nesse caso, mesmo que os atributos de entrada não mudem em distribuição, o significado dos rótulos ou a função alvo mudou, invalidando parcialmente o que o modelo aprendeu originalmente.

Esses fenômenos foram observados em diversos cenários do mundo real. Por exemplo, durante a pandemia de COVID-19, muitos modelos preditivos precisaram ser reajustados – o comportamento dos usuários e mercados mudou drasticamente de forma repentina, um caso clássico de drift abrupto. Mesmo mudanças graduais são problemáticas: modelos de séries temporais podem degradar conforme tendências de longo prazo (ex.: mudança de comportamento do consumidor ano a ano). Sem monitoramento, tais degradações podem passar despercebidas até virarem um problema grave.

Em resumo, monitorar modelos é necessário para identificar model drift, data drift e outros problemas (ex.: dados faltantes, erros de integração) antes que causem impacto. A monitoração efetiva envolve tanto métricas de qualidade do modelo quanto métricas de operação do serviço. Veremos a seguir que tipos de métricas acompanhar e como fazê-lo.

Que métricas acompanhar? Definindo indicadores de saúde do modelo

Métricas de qualidade preditiva: são as métricas que refletem diretamente o desempenho do modelo em sua tarefa. Por exemplo, para um modelo de classificação binária (como nosso modelo de doença cardíaca), podemos acompanhar acurácia, precisão, recall, F1-score, entre outros. Uma queda significativa nesses indicadores ao longo do tempo é o sinal mais direto de model drift. Sempre que possível, é recomendável ter um fluxo para calcular essas métricas usando dados com rótulo real.

Em alguns casos, os rótulos verdadeiros chegam com atraso – por exemplo, modelos médicos podem receber confirmações diagnósticas somente semanas depois ou um modelo de previsão de vendas só sabe o erro após o período previsto passar. Ainda assim, vale a pena calcular a métrica de erro assim que os ground truth estiverem disponíveis, possibilitando detectar uma deterioração na performance assim que ela ocorre.

Métricas de data drift: são indicadores estatísticos de que os dados de entrada ou saída mudaram de distribuição. Mesmo sem rótulos, podemos monitorar se a distribuição das features de entrada se desviou daquela do conjunto de treinamento (data drift) ou se a distribuição das predições do modelo mudou significativamente (prediction drift). Por exemplo, se historicamente seu modelo de doença cardíaca classificava 10% dos pacientes como “alto risco” e de repente passou a classificar 30%, isso pode indicar que algo mudou – talvez os pacientes atendidos mudaram de perfil demográfico ou algum bug passou a alimentar dados errados etc.

Comparar distribuições de probabilidades previstas ao longo do tempo (via testes estatísticos como Kolmogorov-Smirnov, testes qui-quadrado ou métricas de distância como Wasserstein, PSI etc.) ajuda a sinalizar mudanças não esperadas. Essas detecções atuam como “alarme precoce”: podem não confirmar que a acurácia caiu, mas apontam uma mudança ambiental que pode levar a queda de performance. Ferramentas especializadas (como Evidently, WhyLogs etc.) fornecem relatórios automatizados de drift de dados e predições para esse fim.

Métricas de negócio (uso/engajamento): além das métricas puramente técnicas, muitas vezes monitora-se indicadores de uso que, embora não sejam exatamente performance do modelo, correlacionam-se com ela. Por exemplo, em um sistema de recomendação de produtos, pode-se acompanhar a taxa de cliques ou

conversão gerada pelas recomendações como direcionador da qualidade do modelo – se os usuários deixam de clicar nas recomendações, talvez elas tenham ficado irrelevantes, indicando model drift.

No nosso caso de detecção de doença cardíaca, poderíamos monitorar, por exemplo, quantos usuários estão aceitando as recomendações do modelo (se houver interação humana) ou a proporção de positivos prevista. Estes KPIs de produto ajudam a ver se o modelo ainda agrega valor. Outro exemplo é acompanhar a aparição de categorias inéditas em variáveis de entrada – se de repente surgem valores que nunca apareceram no treino (por ex., um novo tipo de exame cardíaco), é um indicativo de drift.

Métricas de serviço (latência, throughput, erros): não podemos esquecer que um modelo em produção faz parte de um sistema de software. Portanto, métricas tradicionais de observabilidade também se aplicam: monitore o tempo de resposta médio e percentil (ex. P95) das inferências para garantir que o modelo atende aos requisitos de desempenho em produção. Uma elevação na latência pode indicar problemas de escala (ex.: aumento de carga) ou problemas no código/modelo (ex.: ficou mais complexo após atualização). Monitore o throughput ou número de requisições atendidas – isso indica o nível de uso do modelo.

Uma queda repentina no número de requisições pode sinalizar uma falha upstream (por exemplo, o sistema que consome o modelo pode ter parado de enviar dados) ou que usuários pararam de usar, o que também merece investigação. E claro, monitore erros de sistema: taxas de erro HTTP (500, timeout) na API de inferência, uso de memória/CPU da instância que hospeda o modelo etc., para identificar problemas operacionais. Essas métricas garantem que o serviço de ML está disponível e estável, complementando as métricas de acurácia. Ferramentas de monitoramento tradicionais podem ser usadas aqui, mas muitas plataformas de MLOps já integram ambas camadas.

Resumindo, devemos definir um conjunto de métricas de acompanhamento abrangente que cubra: qualidade preditiva do modelo, sinais de drift de dados, engajamento/uso e saúde do sistema de inferência.

Ferramentas de monitoramento e alertas

Várias ferramentas estão disponíveis para implementar monitoramento de modelos em produção. A escolha costuma combinar ferramentas generalistas de observabilidade (muito usadas em DevOps) com ferramentas específicas de MLOps. Vejamos algumas populares e como elas se encaixam.

Prometheus & Grafana: uma dupla amplamente utilizada em monitoramento de sistemas. O Prometheus é um toolkit open-source de monitoramento e time-series database, ótimo para coletar e armazenar métricas numéricas ao longo do tempo (com timestamps e labels). Ele funciona através de um modelo pull: periodicamente consulta endpoints expostos por seus serviços monitorados e registra esses valores em sua base temporal. Podemos instrumentar nosso serviço de inferência para expor métricas no formato do Prometheus – por exemplo, usando a biblioteca `prometheus_client` no código Python do Flask, expondo um endpoint `/metrics`.

De fato, no exemplo hands on desta aula, poderíamos expor métricas como `total_predictions` (contador de requisições) e `model_accuracy` (um gauge ou métrica atual de acerto) e o Prometheus coletaria esses dados continuamente (digamos, a cada 15 segundos). Já o Grafana é uma plataforma open-source de observabilidade focada em visualização e alerta. Ele se conecta como front-end ao banco de dados do Prometheus (entre outros possíveis) e permite criar dashboards ricos, gráficos, painéis interativos, além de configurar alertas baseados em consultas nas métricas.

Em outras palavras, o Prometheus cuida da coleta e armazenamento das métricas e o Grafana cuida de exibi-las e disparar alertas. Essa combinação é muito usada em MLOps: por exemplo, podemos ter gráficos de acurácia ao longo do tempo, histogramas de distribuição de uma feature para ver drift, contadores de erro, tudo no Grafana, e notificações (e-mail, Slack etc.) se alguma métrica ultrapassar limites (como latência média $> X$ segundos, ou acurácia $< Y\%$). A força dessa solução é sua robustez e flexibilidade – praticamente qualquer métrica pode ser monitorada. No entanto, configurações customizadas são necessárias para métricas de ML (você precisará programar o logging dessas métricas e possivelmente criar jobs para calcular algumas offline).

MLflow Tracking e Model Registry: ferramentas mais específicas de MLOps integradas ao nosso fluxo de desenvolvimento. Como vimos anteriormente, o MLflow

Tracking é um serviço para logar parâmetros, métricas e artefatos de execuções de ML. Originalmente concebido para experimentos de treinamento, ele também pode ser adaptado para monitorar modelos pós-deployment. Uma abordagem comum é usar o MLflow Tracking em jobs periódicos de avaliação: por exemplo, diariamente você roda um script que carrega o modelo em produção e o testa em um lote de dados novos rotulados, logando a acurácia, AUC etc., como um run no MLflow.

Isso proporciona um histórico de desempenho armazenado que pode ser visualizado via UI do MLflow ou extraído para relatórios. Outra funcionalidade importante é o Model Registry do MLflow, que atua como um repositório central de modelos com controle de versões e estágio. Também vimos anteriormente que, nele, podemos registrar modelos treinados e atribuir estados como “Staging” ou “Production”, facilitando a governança de qual versão do modelo está implantada. No contexto de manutenção, o Model Registry nos permite, por exemplo, comparar versões anteriores do modelo (saber exatamente com qual data e parâmetros cada versão foi treinada) e retroceder (roll back) caso uma versão nova apresente problemas.

Além disso, o registry habilita workflows de promoção controlada – podemos configurar que, quando uma nova versão é registrada como “Staging” e passa em testes, ela seja automaticamente promovida a “Production”. No Databricks, essa promoção pode até acionar a reimplantação se estiver usando Model Serving. Em suma, o MLflow Tracking/Registry serve como base de logging e gestão, complementando ferramentas de métricas em tempo real.

Plataformas de terceiros (observabilidade de ML): nos últimos anos, surgiram diversas ferramentas especializadas em monitoramento de modelos: Arize AI, Evidently, WhyLogs, entre outras. Essas ferramentas focam em fornecer detecção de drift, monitoramento de performance e explicabilidade prontos para uso, integrando dados de predições e (quando disponíveis) rótulos verdadeiros. Por exemplo, o Evidently, oferece pacotes Python para gerar relatórios de data drift, model drift, e métricas de performance que podem ser integrados a pipelines ou dashboards.

Para ilustrar, poderíamos usar o Evidently em nosso caso para criar um relatório semanal comparando a distribuição dos atributos dos pacientes dessa semana vs. da semana em que o modelo foi treinado – se o relatório apontar um

aumento de distância em alguma feature (ex.: média da idade dos pacientes aumentou significativamente), isso seria um sinal de alerta de data drift.

Cada ferramenta tem suas vantagens; o importante é que existem soluções sob medida para monitoramento de modelos de ML que abstraem muita complexidade estatística e oferecem uma interface amigável (muitas vezes com painéis e alertas prontos). Vale mencionar também que grandes provedores de nuvem incorporaram esses recursos em seus serviços de ML: o Azure Machine Learning, por exemplo, oferece o Model Monitor que calcula métricas de drift entre dados de treino e dados de serviço e permite configurar alertas automáticos quando a divergência ultrapassa certo nível. O AWS SageMaker também tem o Model Monitor para estatísticas de dados e endpoint logging. Ou seja, há uma convergência no sentido de que monitoramento de modelo é uma peça fundamental de qualquer plataforma de ML moderna.

Logging estruturado e rastreamento: uma peça muitas vezes negligenciada é o logging detalhado das predições e dados de entrada. Quando possível (respeitando privacidade e volume), é recomendável registrar amostras das requisições recebidas e das respostas do modelo. Por exemplo, logar cada requisição com um ID, timestamp, inputs (pode ser um hash ou alguns atributos-chave, para não armazenar dados sensíveis) e a predição gerada, bem como o resultado real quando disponível.

Esses logs brutos permitem, em caso de detecção de problema, investigar a fundo o que pode ter causado – por exemplo, identificar que todas as predições incorretas vinham de um certo hospital ou de um certo intervalo de datas, sugerindo um problema específico. Ferramentas de log management (ELK stack, Datadog etc.) podem ser usadas para armazenar e consultar esses registros. Além disso, manter um trace de requisições pode ajudar a correlacionar falhas do modelo com outros eventos do sistema (ex.: uma mudança de versão de um serviço upstream). Em suma, logs são dados – e dados ajudam a entender os porquês por trás das métricas.

Alertas e automação: monitorar não adianta se ninguém olhar. Portanto, integrar alertas automáticos é imprescindível. Como citado, o Grafana pode disparar alertas via canais diversos. Podemos configurar, por exemplo: “se a acurácia calculada no job diário for menor que 0.8, enviar e-mail para os engenheiros”; ou “se o desvio entre distribuição atual e de treinamento (medido via teste estatístico) for

significativo ($p\text{-valor} < 0.01$) por 3 dias seguidos, gerar um alerta no Jira”. Em ferramentas cloud, podemos acionar webhooks ou funções (por exemplo AWS Lambda, Azure Functions, etc.) quando detectado drift.

O importante é que haja um processo de resposta: quando um alerta dispara, o time deve revisar o modelo e tomar ação (ou os pipelines já tomam uma ação automática – veremos a seguir sobre retreinamento automático). Alertas demais também é ruim (podem gerar fadiga e serem ignorados), então calibrar bem os thresholds é uma boa prática – muitas equipes começam coletando métricas e analisando manualmente por um período para entender a variabilidade normal, antes de definir thresholds de alerta.

Model Registry e gestão de versões

Conforme várias versões do modelo de ML são produzidas (seja por retreinamentos programados, melhorias do algoritmo ou ajustes após identificar problemas), torna-se fundamental ter controle de versão e um registro central. Isso garante organização e reprodutibilidade, além de facilitar rollback se necessário. Já falamos sobre o MLflow Model Registry como uma solução para isso. Vamos nos aprofundar um pouco mais em seu papel na manutenção de modelos:

O Model Registry permite registrar cada modelo treinado com um nome e acompanhar suas versões sequencialmente, armazenando metadados como fonte do treinamento (experimento MLflow run), parâmetros, quem registrou e quando. Cada versão pode ser anotada e taguada – por exemplo, marcamos versão 1 como “Baseline treinado com dados até 2023”, versão 2 como “Retrained após drift em 2024” etc. Mais importante, podemos atribuir um estado à versão: “Production”, “Staging”, “Archived”.

Assim, quando decidimos que uma nova versão está pronta para uso, mudamos seu estágio para Production e potencialmente rebaixamos a anterior (por exemplo, para Archived). Esse processo de promover modelos entre ambientes fica auditável e controlado pelo Registry. Em grandes empresas, isso vem acompanhado de governança – somente usuários autorizados podem marcar um modelo como produção, muitas vezes após passar por checklists ou validações.

Na manutenção contínua, o registro de modelos é a peça central para orquestrar atualizações sem caos. Imagine que detectamos que o modelo atual (versão 3) deteriorou. Decidimos retreinar: isso resultará na versão 4. No Registry, podemos deixar a versão 3 marcada como Production até testarmos a 4. Podemos carregar facilmente a versão 4 em um ambiente de Staging (por exemplo, usando URI `models:/HeartDiseaseModel/4` ou por alias) e rodar testes comparativos. Ferramentas como o MLflow oferecem a capacidade de fazer A/B testing ou shadow deployments usando essas versões – por exemplo, o alias `Champion/Challenger` pode ser usado para trafegar parte do tráfego para a nova versão enquanto monitoramos resultados. Somente quando confiarmos que a nova versão é superior, promovemos 100% para ela.

Outro benefício é que, caso a versão nova apresente algum comportamento inesperado em produção, conseguimos rapidamente reverter para a versão anterior. Como as versões antigas ficam arquivadas no Registry, basta mudar o estágio da v3 de volta para Production e fazer um novo deployment. Assim minimizamos downtime e impactos. Essa segurança de rollback encoraja a atualização frequente, pois temos um plano B fácil caso algo dê errado.

Além do MLflow, existem outras soluções de registro e versionamento: por exemplo, o Kubeflow tem seu conceito de versão de modelo e empresas como Netflix e Uber desenvolveram seus próprios sistemas internos de registro e implantação com versão (muitas vezes integrados ao CI/CD padrão). O importante é que versionar modelos (e idealmente também versionar os dados de treinamento utilizados e o código) é parte crucial do ciclo MLOps – isso evita “perder de vista” o que está rodando. Vale lembrar que um modelo não versionado é um risco: se alguém treina uma nova versão e substitui a antiga sem registro, perdemos a referência para investigar problemas ou reproduzir resultados.

Em suma, faça do Model Registry seu aliado na manutenção: use-o para documentar evoluções do modelo, comparar performance entre versões (o MLFlow inclusive pode armazenar métricas de avaliação por versão) e controlar implantações gradativas.

Estratégias de manutenção contínua: quando e como atualizar seu modelo

Agora que sabemos monitorar e temos infraestrutura para gerir versões, falta responder: quando acionar uma manutenção (retreinamento) do modelo? Existem diferentes estratégias, que podem ser combinadas conforme o caso:

- **Agendamento periódico de re-treinamento:** a abordagem mais simples é estabelecer um cronograma fixo – por exemplo, re-treinar o modelo a cada semana ou mês com os dados mais recentes. Essa estratégia garante que, mesmo sem sinais aparentes de problema, o modelo é atualizado regularmente, prevenindo acúmulo de drift. No entanto, é importante alinhar a frequência à dinâmica do problema. Se o evento modelado muda rapidamente (ex.: detecção de fraudes, onde fraudadores inovam constantemente), retrainar diariamente ou semanalmente pode ser necessário. Já em cenários mais estáveis (ex.: um modelo de previsão de manutenção de máquinas industriais), retreinamentos trimestrais podem ser suficientes. O perigo de retrainar por intervalo arbitrário é gastar recursos e possivelmente introduzir ruído se não houver mudança significativa – por isso, muitas empresas escolhem um intervalo baseado em observação ou em requisitos de negócio.
- **Gatilho por degradação de performance:** consiste em monitorar métricas de performance (ex.: acurácia, erro médio) e disparar retreinamento assim que o desempenho cair abaixo de um limiar pré-definido. Por exemplo, se a acurácia em produção (medida pelas amostras rotuladas mais recentes) ficar abaixo de 85%, inicia-se automaticamente um job de retreino. Essa abordagem garante que o modelo seja atualizado somente quando necessário, ou seja, quando há evidência de que ele não atende mais ao nível de serviço esperado. Ela requer que tenhamos um fluxo para obter métricas de acurácia continuamente – e muitas vezes espera-se pelo ground truth, o que pode causar um delay. Em problemas onde o rótulo demora a chegar (ex.: modelo de crédito que só sabe se o cliente “negativou” após meses), essa estratégia pode reagir tarde demais. Ainda assim, para casos de feedback rápido (ex.: detecção de spam – usuários marcam email como “não spam”, dando rótulo imediato), é bastante efetiva.

- **Gatilho por data drift:** aqui, não esperamos a queda de acurácia comprovada, mas sim usamos detecção de drift nos dados como gatilho. Por exemplo, se nossos monitores de drift detectarem uma mudança significativa na distribuição de entradas ou previsões, podemos acionar um retreinamento automático. A lógica é que se o contexto mudou, provavelmente o modelo precisará se adaptar, então já coletamos novos dados e retreinamos antes mesmo de ver a performance cair muito. Essa estratégia é útil para problemas onde o rótulo é escasso ou atrasado. No entanto, cuidado: nem todo data drift significa que o modelo piorou. Pode ser que o modelo ainda esteja indo bem naquela nova distribuição de dados. Portanto, alguns times combinam essa abordagem com validação manual – detecção de drift aciona um alerta e um cientista decide se retreina ou não. Alternativamente, pode-se integrar o drift como um critério adicional (ex.: drift detectado e persistindo por X dias aciona retreino).
- **Atualização contínua ou online:** em vez de retreinar do zero periodicamente, essa estratégia envolve incorporar os novos dados incrementalmente ao modelo de forma contínua (aprendizado online) ou quase contínua. Por exemplo, cada novo dado recebido (com rótulo) alimenta um processo que refina o modelo. Isso é complexo de implementar e nem todos os algoritmos suportam bem, mas em certos casos é usado – por exemplo, modelos de recomendação podem ser atualizados com interações do usuário em tempo quase-real ou modelos de detecção de anomalias que atualizam seu baseline constantemente. A vantagem é que o modelo se adapta rapidamente; a desvantagem é o risco de instabilidade sem gerenciamento adequado.
- **Retraining on demand (manualmente):** por fim, sempre há a possibilidade de retreinar manualmente quando a situação exigir. Por exemplo, se ocorreu um evento extraordinário (pandemia, mudança regulatória) que sabemos que invalida o modelo atual, o time pode decidir retreinar fora do calendário. Essa categoria inclui quando se descobre um problema no modelo (ex.: viés identificado) e então coleta-se mais dados ou ajusta-se o modelo lançando uma nova versão. Basicamente, mantenha a flexibilidade para intervenções human-in-the-loop quando preciso.

Na prática, muitos fluxos de MLOps combinam agendamento regular + monitoração de triggers. A Netflix, por exemplo, reporta que monitora vários sinais (qualidade preditiva, deriva de features, métricas de negócio) para identificar quando um modelo precisa de intervenção ou retraining. Em último caso, se um modelo degrada a ponto de ser inutilizável e não há tempo hábil para retreinar, equipes devem ter planos de contingência – como reverter para um modelo anterior que embora antigo estava funcionando melhor ou usar um modelo mais simples de backup. Ter um modelo de backup pode ser útil em cenários críticos (ex.: se um modelo complexo falhar, um modelo linear mais simples ainda opera, talvez com menor acurácia mas sem erro total).

Uma vez decidido que vamos retreinar, é importante também pensar como automatizar o pipeline de retreinamento. Ferramentas de orquestração (Airflow, Kubeflow Pipelines, Databricks Jobs etc.) podem ser configuradas para executar todo o fluxo: ingestão de novos dados, pré-processamento, treinamento do modelo atualizado, avaliação, registro no Model Registry e implantação. Assim, o loop fica fechado sem necessidade de intervenção manual, o que é o ideal em um ambiente de MLOps maduro. Claro, deve-se incluir etapas de validação nesse pipeline (testes automatizados no novo modelo, comparação com antigo etc.) para não implantar algo pior por acidente.

Resumindo, mantenha políticas claras de manutenção: defina qual critério deve acionar uma atualização, com que frequência mínima ocorrem retreinamentos e implemente automação para executá-los e implantar novos modelos com segurança. Isso garante que seus modelos não “degradem” em produção e reduz o trabalho manual.

MERCADO, CASES E TENDÊNCIAS

Para consolidar, vale olhar para algumas **lições práticas** de casos reais de deterioração de modelos e como foram tratados:

- **Pandemia e mudanças bruscas:** muitos setores tiveram seus modelos de Machine Learning (ML) impactados durante a COVID-19. Por exemplo, modelos de demanda em companhias aéreas, modelos de risco de crédito em bancos e modelos de manutenção preditiva em fábricas tiveram que ser rapidamente refeitos quando padrões históricos perderam relevância. A lição aqui é: esteja preparado para eventos fora do comum. Modelos muito ajustados a padrões históricos podem falhar feio sob condições inéditas. Boas práticas incluem: projetar o pipeline de retreino para ser o mais ágil possível (para reagir rápido) e talvez manter algumas reservas de capacidade para casos de retreinamento emergencial (dados já prontos, infraestrutura disponível). Algumas empresas adotaram modelos híbridos ou com componentes adaptativos para tentar reagir melhor a outliers – por exemplo, sistemas de recomendação que aumentam o peso de tendências atuais de forma temporária.
- **Drift lento e imperceptível:** um case interessante foi relatado na área de saúde: pesquisadores observaram que modelos de diagnóstico por imagem treinados em um hospital perdiam acurácia quando usados anos depois ou em outro hospital, devido a mudanças sutis nos equipamentos e na população de pacientes – um caso de drift gradual. Em um estudo de 2023, concluiu-se que data drift é um dos maiores causadores de deterioração de performance em ML médico e enfatizou-se a necessidade de identificar drift cedo e aplicar estratégias de mitigação, como retreino periódico e técnicas de adaptação de domínio. Isso reforça a importância de monitorar continuamente mesmo quando aparentemente "tudo está bem". Muitas vezes, a acurácia pode estar caindo 0,1% a cada semana – imperceptível em curto prazo – mas depois de meses o acumulado é grande. Ferramentas de monitoramento podem plotar essas tendências e permitir intervir antes que o estrago seja grande.

- **Falhas por falha nos dados ou pipelines:** nem toda queda de performance vem de drift genuíno; às vezes, é bug. Por isso, monitore também a integridade dos dados (quantidade de nulos, intervalos válidos etc.). Empresas como Uber e Airbnb têm camadas de validação de dados em tempo real (Great Expectations, por exemplo) para assegurar que o modelo não consuma dados ruins. Quando um modelo apresenta comportamento estranho, além de retreinar, sempre verifique se algo mudou na geração dos dados de entrada ou no código de feature engineering.
- **Estratégias de defesa:** grandes empresas montam verdadeiros “observability teams” para ML. A Netflix, por exemplo, implementou monitoramento extensivo nos seus modelos de recomendação, acompanhando desde métricas online (cliques, retenção) até feature drift e métricas de qualidade de recomendação, tudo para decidir se um modelo precisa ser ajustado ou não. Eles também praticam implantação gradual (canary releases): quando lançam um modelo novo, primeiramente direcionam uma pequena fração de usuários a ele e comparam métricas de engajamento e satisfação vs. o modelo antigo. Se algo estiver pior, abortam o lançamento – assim evitam falhas massivas. Essa é uma ótima prática de manutenção: testar em produção em pequena escala antes de trocar completamente. Da mesma forma, o Uber relatou que seus pipelines de ML têm camadas de redundância – se um modelo em tempo real falhar, entra um modelo de backup ou até regras manuais temporariamente (ex.: se o modelo de ETA de viagens cair, usam uma fórmula básica calculada pela distância). Isso garante resiliência enquanto o time consegue resolver o problema do modelo principal.
- **Ferramentas e Plataformas Emergentes:** nos últimos anos, vimos uma onda de plataformas dedicadas à observabilidade de ML. Por exemplo, a Arize AI e a Fiddler AI oferecem soluções completas para monitorar data drift, performance de modelos e melhorar a explicabilidade dos modelos, com adoção em setores financeiros e de tecnologia. Já o Evidently AI (open source) tem sido utilizado vastamente, mostrando a crescente demanda por ferramentas práticas para melhorar o monitoramento e a observabilidade.

Grandes provedores em nuvem também incorporaram essas funcionalidades – em 2023, a Microsoft anunciou o Azure ML Model Monitor, que unifica monitoramento de data drift com métricas customizadas e integra com alertas via Application Insights. A tendência é que o monitoramento de modelos se torne padrão assim como já temos com monitoramento de servidores no geral.

- **COVID-19 como catalisador:** o choque da pandemia serviu de aprendizado para muitos times de ML. Uma leitura recomendada é o artigo “How Instacart’s Item Availability Evolved Over the Pandemic” – onde os autores descrevem os desafios enfrentados quando seus modelos de previsão tiveram que lidar com mudanças abruptas de demanda e estoque e como solucionaram com novas arquiteturas e limiares adaptativos. De modo geral, pós-2020 houve maior conscientização sobre resiliência de modelos: empresas passaram a exigir que modelos críticos tenham planos de contingência e sejam monitorados para reagir a eventos inesperados.
- **Tendências Futuras:** olhando para frente, espera-se maior integração entre monitoramento automatizado e ação autônoma. Em outras palavras, sistemas de MLOps que não apenas disparam alertas, mas que possam auto-reconfigurar modelos diante de anomalias ou rodar automaticamente um pipeline de retreino e validação (talvez com aprovação humana mínima). Isso se alinha ao conceito de AutoML contínuo. Além disso, com a popularização de modelos de grande porte (ex.: modelos de linguagem), surgem discussões sobre como monitorar coisas como data drift em modelos de linguagem (LLMs) e alucinações. Já existem ferramentas emergentes para observar LLMs em produção (ex.: LangChain e MLFlow 3.0). Portanto, o escopo do monitoramento está se expandindo.

Vale mencionar também que há um movimento forte rumo à Observabilidade de ML como disciplina própria. Novas startups e ferramentas focam em facilitar essa parte do ciclo de vida. Além disso, discussões sobre regulamentações de IA (como a AI Act na União Europeia) têm enfatizado que sistemas de IA de alto risco precisarão ter monitoramento, log de decisões e planos de atualização – ou seja, a própria legislação pode exigir práticas de manutenção robustas. Isso provavelmente impulsionará a adoção dessas práticas mesmo em empresas que ainda não as fazem.

A mensagem é clara: não existe modelo “fire-and-forget”. A manutenção e o monitoramento são parte integrante do ciclo de vida de modelos. Com as ferramentas certas, métricas bem escolhidas e planos de ação (retreino, versionamento, rollback), conseguimos antecipar problemas ou corrigi-los rapidamente, evitando surpresas desagradáveis e mantendo a confiança nos sistemas de IA. Em suma, o mercado e a comunidade estão convergindo para tornar o monitoramento e a manutenção partes inseparáveis do ciclo de vida de ML. Os(as) profissionais capazes de implementar boas práticas de MLOps – incluindo monitoramento – estão em alta demanda, pois garantem que investimentos em IA realmente se traduzam em valor contínuo, não em sistemas que falham sem aviso.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos a fase de monitoramento e manutenção de modelos de ML em produção, completando o ciclo pós-implantação. Você aprendeu por que modelos degradam com o tempo e conheceu conceitos fundamentais como data drift e model drift, entendendo como eles impactam a performance de um modelo ao longo de sua vida útil. Discutimos quais métricas monitorar para acompanhar a saúde de um modelo – desde métricas preditivas (acurácia, erro) até métricas de serviço (latência, disponibilidade) – e como definir alertas para mudanças preocupantes nesses indicadores.

Vimos também as principais ferramentas de monitoramento utilizadas em MLOps: o uso combinado de Prometheus e Grafana para coletar e visualizar métricas em tempo real, bem como o MLflow para logging e gestão de versões via Model Registry. Abordamos a importância de manter um registro de modelos versionado, possibilitando rastreabilidade, comparações e rollbacks seguros durante a manutenção contínua. Além disso, examinamos diferentes estratégias de manutenção: retreinamentos periódicos agendados vs. acionados por degradação detectada e como automatizar pipelines de atualização de modelos.

Por fim, analisamos casos reais e melhores práticas de mercado – de como empresas reagiram a cenários de grande mudança (como a pandemia) até a institucionalização de observabilidade de ML em organizações como Netflix. Com isso, reforçamos a ideia de que implantar um modelo é apenas o começo: mantê-lo relevante e confiável requer monitoramento diligente e ajustes contínuos. Você sai desta aula equipado(a) com conhecimentos e exemplos práticos para implementar um robusto processo de monitoramento e manutenção, garantindo que seus modelos de ML continuem gerando valor e insights de forma sustentada.

REFERÊNCIAS

AZURE ML DOCUMENTATION. **Monitor data drift**. 2025. Disponível em: <https://learn.microsoft.com>. Acesso em: 28 jan. 2026.

BRAINFOGE. **How Netflix Uses ML to Create Recommendations**. 2023. Disponível em: <https://www.brainforge.ai/blog/how-netflix-uses-machine-learning-ml-to-create-perfect-recommendations>. Acesso em: 28 jan. 2026.

DATA CAMP. **Grafana Tutorial: A Beginner's Guide to Monitoring Machine Learning Models**. 2023. Disponível em: <https://www.datacamp.com/tutorial/grafana-tutorial-monitoring-machine-learning-models>. Acesso em: 28 jan. 2026.

DATADOG. **Machine learning model monitoring: Best practices**. 2024. Disponível em: <https://www.datadoghq.com/blog/ml-model-monitoring-in-production-best-practices/>. Acesso em: 28 jan. 2026.

EVIDENTLY AI. **Model monitoring for ML in production: a comprehensive guide**. 2025. Disponível em: <https://www.evidentlyai.com/ml-in-production/model-monitoring>. Acesso em: 28 jan. 2026.

GOUABOU, A. **Monitoring ML Model Performance with Prometheus & Grafana**. 2025. Disponível em: <https://medium.com/@cartelgouabou/step-by-step-guide-monitoring-ml-model-performance-with-prometheus-grafana-88195f741365>. Acesso em: 28 jan. 2026.

IBM DOCUMENTATION. **Collecting telemetry data using Prometheus – Grafana**. 2025. Disponível em: <https://www.ibm.com/docs/en/watsonxdata/standard/2.1.x?topic=v211-collecting-telemetry-data-using-prometheus-grafana>. Acesso em: 28 jan. 2026.

INSTACART – TECH BLOG. **How Instacart's Item Availability Evolved Over the Pandemic**. 2023. Disponível em: <https://www.instacart.com/company/tech-innovation/how-instacarts-item-availability-evolved-over-the-pandemic>. Acesso em: 28 jan. 2026.

MLFLOW DOCUMENTATION. **MLflow Model Registry**. 2025. Disponível em: <https://mlflow.org>. Acesso em: 28 jan. 2026.

NEPTUNE.AI. **Retraining Model During Deployment: Continuous Training and Continuous Testing**. 2022. Disponível em: <https://neptune.ai/blog/retraining-model-during-deployment-continuous-training-continuous-testing>. Acesso em: 28 jan. 2026.

NETFLIX TECH BLOG. **ML Observability: Bring Transparency to Payments and Beyond**. s.d. Disponível em: <https://netflixtechblog.com/ml-observability-bring-transparency-to-payments-and-beyond-33073e260a38>. Acesso em: 28 jan. 2026.

QWAK. **Top 7 ML Model Monitoring Tools in 2024**. 2024. Disponível em: <https://www.qwak.com/post/top-ml-model-monitoring-tools>. Acesso em: 28 jan. 2026.

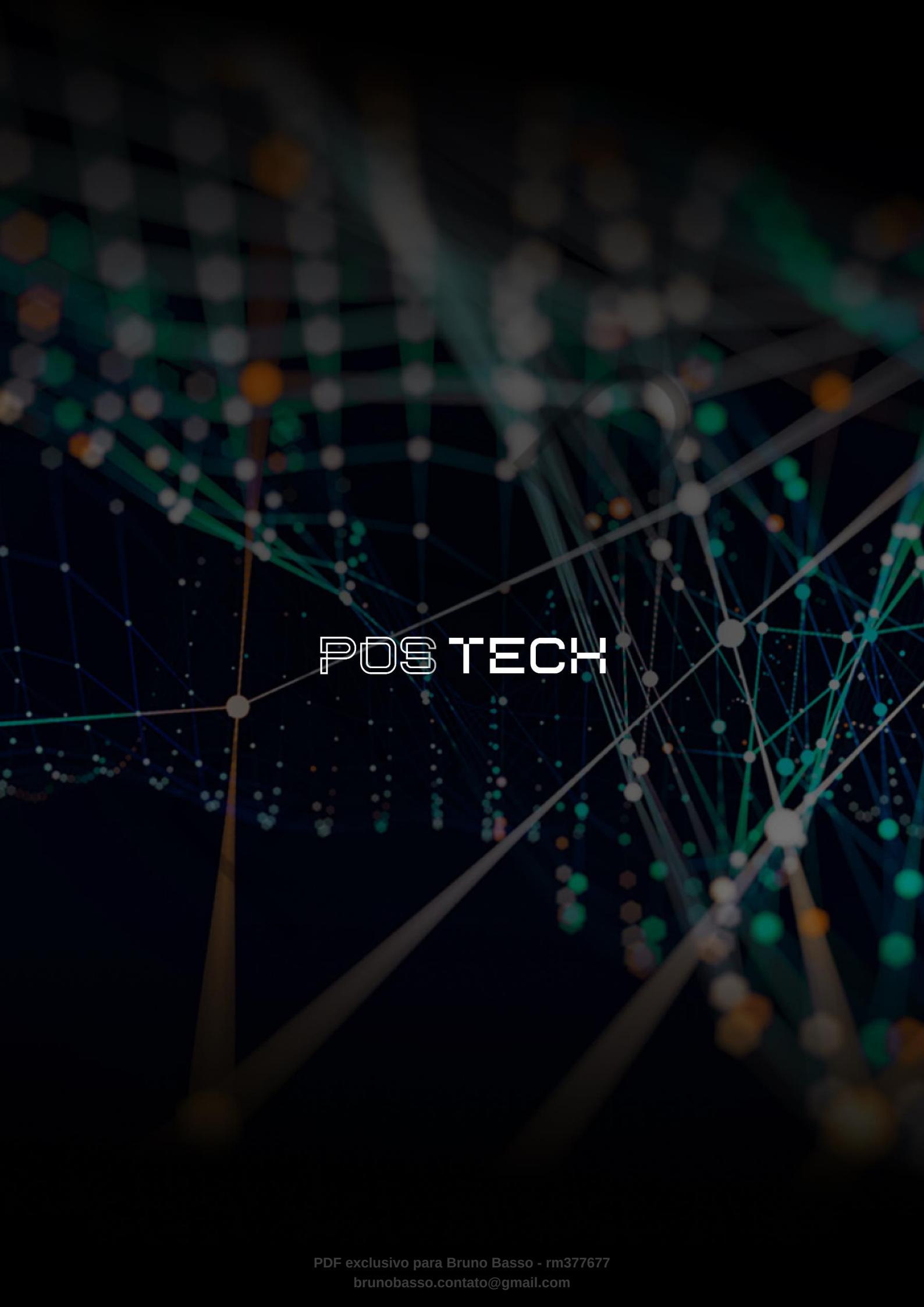
SAHINER, B.; CHEN, W.; SAMALA, R. K.; PETRICK, N. **Data drift in medical machine learning: implications and potential remedies**. 2023. Disponível em: <https://pmc.ncbi.nlm.nih.gov>. Acesso em: 28 jan. 2026.

EMEND

PALAVRAS-CHAVE

Monitoramento. Manutenção. Data Drift.

EMSE



POSTECH